

# Урок 2. Netty Request Runtime

## 0. Что слушатель поймет после урока

- что такое Netty server в WebFlux-приложении;
- что такое `Channel``;
- что делает `ChannelPipeline``;
- `EventLoopGroup`` / `EventLoop``;
- Разбор практического Backend-сценария;

## 1. Зачем нужен второй урок

На первой лекции мы разобрали то, зачем вообще нужна reactive/non-blocking модель:

Вспомнили Spring MVC - Blocking I/O модель и закрепили главные мысли:

```
blocking I/O удерживает поток ожиданием
↓
много параллельных I/O waiting операций требуют много platform threads
↓
много platform threads стоят памяти, scheduler overhead и context switches
↓
non-blocking model позволяет не держать поток во время ожидания внешнего I/O
```

Мы также ввели ключевую идею:

```
Один EventLoop может обслуживать много Channel-ов.
Внутри одного EventLoop выполнение последовательное.
```

## 2. Уточнение после первого урока: EventLoop работает последовательно

Когда мы говорим:

```
EventLoop обрабатывает события последовательно
```

Это не значит:

```
Если в систему поступило два HTTP-запроса, то первый HTTP-запрос обязан полностью завершиться раньше второго HTTP-запроса - НЕВЕРНО
```

Более того, это не значит:

```
EventLoop держит request-1 до полного response, и только потом начинает request-2 - НЕВЕРНО
```

Правильнее всего говорить так:

EventLoop последовательно выполняет короткие события и callback'и, но разные HTTP-запросы могут продвигаться interleaving'ом и завершаться в другом порядке.

**Interleaving** означает, что выполнение разных запросов перемешивается во времени в рамках одного потока исполнения:

```
кусок request-1
кусок request-2
продолжение request-2
продолжение request-1
```

## 2.1. Поглубже про Interleaving

Объяснить.

### Аналогия из жизни: Шахматный гроссмейстер

Представьте гроссмейстера, который проводит сеанс одновременной игры против 10 любителей(10 внешних сервисов).

**Блокирующий подход (Thread-per-request) - нет Interleaving'a**

**Реактивный подход с Interleaving**

**Движение гроссмейстера между досками — это и есть interleaving**

## 3. Пример endpoint-а из первой лекции

Возьмем endpoint из примеров первой лекции:

```
@GetMapping("/profile/{id}")
fun profile(@PathVariable id: String): Mono<ProfileResponse> {
    return Mono.just(id)
        .map { validate(it) }
        .flatMap { validId ->
            externalClient.getProfile(validId)
        }
        .map { externalResponse ->
            ProfileResponse.from(externalResponse)
        }
}
```

Здесь есть три смысловых участка:

|                                      |                                                 |
|--------------------------------------|-------------------------------------------------|
| 1.validate(id)                       | — короткая синхронная работа                    |
| 2.externalClient.getProfile(validId) | — внешний non -blocking I / O вызов             |
| 3.ProfileResponse.from(response)     | — продолжение реактивного pipeline после ответа |

Ключевой момент находится во втором пункте.

Если `externalClient.getProfile(...)` построен на non-blocking клиенте, например `WebClient`, то EventLoop не обязан стоять и ждать внешний сервис.

### 3.1. Пример с двумя запросами

Допустим, почти одновременно пришли два запроса:

```
GET /profile/1
GET /profile/2
```

На входе у нас появились два inbound Channel:

```
Client-1 -> Channel-A
Client-2 -> Channel-B
```

Пусть оба Channel попали на один EventLoop:

```
EventLoop-1
├─ Channel-A: /profile/1
└─ Channel-B: /profile/2
```

ВАЖНО, еще раз:

```
EventLoop последовательно выполняет отдельные события и callback'и.
Но это не означает, что HTTP-запросы обязаны завершаться строго в порядке прихода
```

### 3.2. Что значит "последовательно"

Это не значит:

```
request-1 обязан полностью завершиться раньше request-2 - НЕВЕРНО
```

И это не значит:

```
EventLoop держит request-1 до полного HTTP response и только потом начинает request-2 - НЕВЕРНО
```

Для non-blocking WebFlux flow с внешним I/O картина другая.

Когда pipeline доходит до:

```
externalClient.getProfile(validId)
```

EventLoop инициирует внешний non-blocking call, регистрирует продолжение(callback) и возвращается к другим событиям.

### 3.3. Хронология(Timeline) обработки запросов

Сначала EventLoop принимает первый запрос:

```
t1:
  входящий event: /profile/1
  validate(1)
  неблокирующий внешний вызов для request-1
  возврат в EventLoop
```

После этого EventLoop может принять второй запрос:

```
t2:
  входящий event: /profile/2
  validate(2)
  неблокирующий внешний вызов для request-2
  возврат в EventLoop
```

После вызова внешнего сервиса, оба HTTP-запроса будут находиться в состоянии ожидания внешнего I/O:

```
Inbound:
  Channel-A ждет response для Client-1
  Channel-B ждет response для Client-2

Outbound:
  Channel-X(A) ждет response от external-service для request-1
  Channel-Y(B) ждет response от external-service для request-2
```

Допустим, внешний сервис быстрее ответил на второй запрос:

```
t3:
  external-service вернул ответ для request-2
```

Тогда продолжение для второго запроса может выполняться раньше:

```
t4:
  продолжение pipeline для request-2
  ProfileResponse.from(response-2)
  write HTTP response to Client-2
```

И состояние будет таковым:

```
Inbound:
  Channel-A ждет response для Client-1
  Channel-B ждет response для Client-2

Outbound:
  Channel-X(A) ждет response от external-service для request-1
```

И, соответственно, ответ вернется к Client-2 и состояние будет:

```
Inbound:
  Channel-A ждет response для Client-1

Outbound:
  Channel-X(A) ждет response от external-service для request-1
```

И только потом может прийти ответ для первого запроса:

```
t5:
  external-service вернул ответ для request-1

t6:
  продолжение pipeline для request-1
  ProfileResponse.from(response-1)
  отдать HTTP ответ для Client-1
```

Итоговый порядок:

```
Request order:
  1. Client-1
  2. Client-2

Response order:
  1. Client-2
  2. Client-1
```

### 3.4. Главная формулировка

Последовательное исполнение событий не равно последовательному завершению запросов

Еще одна важная формулировка:

Внутри одного EventLoop нет параллельного выполнения событий, но есть interleaving(чередование/реагирование) этих самых событий

Лучше всего говорить так:

EventLoop последовательно обрабатывает короткие события и callback'и, но разные запросы могут продвигаться interleaving'ом и завершаться в другом порядке.

Финальная формулировка:

В одном EventLoop нет параллельного исполнения callback'ов. Но есть конкурентное продвижение многих запросов через события. Порядок завершения запроса определяется тем, какие I/O-события пришли раньше, а не только тем, какой HTTP-запрос первым вошел в приложение.

## 4. Главная цель текущего урока

Первый урок отвечал на вопрос:

Зачем нужна reactive/non-blocking модель?

Второй урок отвечает на другой вопрос:

Понять, как HTTP-запрос входит в приложение до уровня Reactor Netty.

Наша задача — научиться понимать и объяснять путь запроса в WebFlux так же, как объясняют путь запроса в Spring MVC:

Spring-MVC:

```
Tomcat
↓
servlet thread
↓
DispatcherServlet
↓
HandlerMapping
↓
HandlerAdapter
↓
Controller
↓
response
```

```
Netty server
↓
Channel
↓
ChannelPipeline
↓
EventLoop
↓
Reactor Netty bridge
↓
Spring WebFlux поверх Netty
↓
HttpHandler / WebHandler
↓
DispatcherHandler
↓
HandlerMapping
↓
HandlerAdapter
↓
Controller returns Mono/Flux
↓
Spring subscribes to Publisher
↓
HTTP response write
```

Сегодня нас пока не интересует подробная механика:

```
flatMap
zip
retryWhen
publishOn
subscribeOn
слияние operator'ов
backpressure strategies
```

Нас интересует именно путь HTTP-запроса.

## 5. Переход к изучению Netty

Начинаем с самого нижнего прикладного уровня.

Запуск Spring Boot WebFlux-приложения

```
слушать порт приложения (8080)
принимать TCP-соединения от клиентов
читать HTTP-запросы
отдавать HTTP-ответы
```

В Spring MVC эту роль часто выполняет servlet container:

```
Spring MVC
↓
Tomcat / Jetty / Undertow
↓
Servlet API
↓
worker thread pool
```

В Spring Boot WebFlux по умолчанию эту роль обычно выполняет Reactor Netty.

Упрощенно стек WebFlux-приложения выглядит так:

```
Spring WebFlux
↓
Reactor Netty
↓
Netty
↓
Java NIO / native transport
↓
OS socket API
```

### 5.1. Определение границ ответственности

На этом уровне важно не запоминать десятки классов, а понять границы ответственности:

```
Spring WebFlux:
  отвечает за веб-слой приложения:
  представляет HTTP-запрос и HTTP-ответ в виде Spring-абстракций,
  прогоняет запрос через цепочку WebFilter-ов,
  через HandlerMapping находит нужный обработчик,
  через HandlerAdapter вызывает controller/handler method,
  получает HandlerResult,
  через HandlerResultHandler обрабатывает результат контроллера
  и передает response дальше в реактивный механизм записи HTTP-ответа.

Reactor Netty:
  связывает Spring WebFlux с Netty HTTP runtime.

Netty(HTTP runtime):
  отвечает за сетевой runtime:
  TCP-connection, Channel, Pipeline, EventLoop, read/write эвенты.
```

```
OS socket API:
    нижний уровень, где живут сокеты.
```

То есть, например, controller — это не первая точка входа, а верхний слой обработки, и до него запрос проходит через сетевой runtime(Netty).

## 6. Что значит “Netty server”

Netty server — это сетевой runtime, который:

- Слушает порт приложения(8080)
- Операционная система принимает TCP-соединения по данному порту
- Netty создает Channel для этого соединения
- Netty назначает Channel на EventLoop
- Прогоняет входящие байты через ChannelPipeline
- Пишет исходящие байты обратно клиенту

Концептуально:

```
Client
  ↓
TCP connection
  ↓
Netty принимает соединение
  ↓
Netty создает Channel
  ↓
Channel регистрируется в EventLoop
  ↓
ChannelPipeline
  ↓
Reactor Netty
  ↓
Spring WebFlux
```

Можно представить так:

```
Spring Boot WebFlux application

| OS Socket (Сетевой сокет ОС)
|   ↓
| Netty Channel (Создание Java-абстракции над сокетом)
|   ↓
| EventLoop / EventLoopGroup -
|   (Регистрация канала на конкретном потоке)
|   ↓
| Netty ChannelPipeline -
|   (Прогон байтов через цепочку фильтров сокета)
|   ↓
| Reactor Netty HttpServer -
|   (Сборка байтов в единый Java-объект HTTP-запроса)
|   ↓
| Spring WebFlux -
|   (Маршрутизация и применение фильтров фреймворка)
|   ↓
```



```
@RestController -
(Выполнение бизнес-логики)
```

И здесь появляется первая важная граница:

Netty не знает про `@RestController`, т.к. всё, что видит Netty — это открытый сетевой порт, сокет, входящие потоки байтов и правила их парсинга по стандарту HTTP. Он мыслит категориями сетевых пакетов, а не бизнес-логики

И Spring WebFlux не работает напрямую с сырыми TCP-байтами — этот слой ожидает, что на вход ему дадут уже готовые, структурированные Java-объекты запроса, из которых можно легко достать JSON, параметры или заголовки.

Reactor Netty берет низкоуровневые данные Netty и переводит их в данные, понятные Project Reactor'у (Mono/Flux).

Вернее, Reactor Netty адаптирует Netty HTTP runtime к reactive API: серверный request/response становятся частью reactive pipeline, а Spring WebFlux получает возможность работать через `HttpHandler`.

## 7. Небольшая практика к пункту Netty server

Пока мы не хотим глубоко лезть в `ChannelPipeline` и собственные Netty handler-ы.

Но уже на первом шаге можно показать, что WebFlux-приложение действительно работает поверх Reactor Netty.

### Анализ логов:

#### 1. Рождение сетевого канала:

```
2026-06-01 18:43:21.230 [reactor-http-nio-3] DEBUG r.n.http.server.HttpServerOperations - [c0fc3f5e,
L:/[0:0:0:0:0:0:1]:8085 - R:[0:0:0:0:0:0:1]:51579] New http connection, requesting read
```

#### 2. Инициализация конвейера фильтров:

- `loggingHandler` — логирует события сети
- `httpCodec` — парсер, который умеет превращать куски байт в структуру HTTP-протокола.
- `reactiveBridge` — как раз тот самый слой Reactor Netty, который стоит в конце конвейера и переводит данные на язык реактивных потоков

```
2026-06-01 18:43:21.230 [reactor-http-nio-3] DEBUG r.netty.transport.TransportConfig - [c0fc3f5e,
L:/[0:0:0:0:0:0:1]:8085 - R:[0:0:0:0:0:0:1]:51579] Initialized pipeline
DefaultChannelPipeline{(reactor.left.loggingHandler =
reactor.netty.transport.logging.ReactorNettyLoggingHandler), (reactor.left.httpCodec =
io.netty.handler.codec.http.HttpServerCodec), (reactor.left.httpTrafficHandler =
reactor.netty.http.server.HttpTrafficHandler), (reactor.right.reactiveBridge =
reactor.netty.channel.ChannelOperationsHandler)}
```

#### 3. Канал готов к работе:

```
2026-06-01 18:43:21.230 [reactor-http-nio-3] DEBUG r.netty.http.server.HttpServer - [c0fc3f5e,
L:/[0:0:0:0:0:0:1]:8085 - R:[0:0:0:0:0:0:1]:51579] REGISTERED
```

```
2026-06-01 18:43:21.230 [reactor-http-nio-3] DEBUG r.netty.http.server.HttpServer - [c0fc3f5e,
L:/[0:0:0:0:0:0:1]:8085 - R:/[0:0:0:0:0:0:1]:51579] ACTIVE
```

#### 4. EventLoop читает входящие байты из Channel:

```
2026-06-01 18:43:21.231 [reactor-http-nio-3] DEBUG r.netty.http.server.HttpServer - [c0fc3f5e,
L:/[0:0:0:0:0:0:1]:8085 - R:/[0:0:0:0:0:0:1]:51579] READ: 82B
```

```
|00000000| 47 45 54 20 2f 74 65 73 74 31 20 48 54 54 50 2f |GET /test1 HTTP/|
|00000010| 31 2e 31 0d 0a 48 6f 73 74 3a 20 6c 6f 63 61 6c |1.1..Host: local|
```

#### 5. Ожидание будущего ответа для запроса:

```
2026-06-01 18:43:21.232 [reactor-http-nio-3] DEBUG r.n.http.server.HttpServerOperations - [c0fc3f5e,
L:/[0:0:0:0:0:0:1]:8085 - R:/[0:0:0:0:0:0:1]:51579] Increasing pending responses count: 1
```

#### 6. Передача эстафеты в Spring WebFlux и освобождение EventLoop:

**Подробнее про работу реактивности в текущем запросе:**

- ОС принимает сетевые данные
- EventLoop получает событие готовности сокета к чтению.
- Поток `EventLoop` считывает эти байты и проталкивает их вверх по конвейеру ChannelPipeline
- HTTP codec превращает сырые байты в объект HTTP request.
- Reactor Netty передает этот request в Spring WebFlux через ReactorHandlerAdapter
- Spring WebFlux доходит до нашего controller method.
- Controller возвращает Mono, который описывает будущую работу.
- После этого controller method завершается.
- Поток `nio-3` освобождается.
- Netty генерирует событие `READ COMPLETE`

**Важно:** READ COMPLETE не означает, что весь бизнес-запрос уже завершился и response отправлен клиенту.

READ COMPLETE означает, что на текущем этапе EventLoop завершил чтение входящих данных из Channel.

Для нас этот лог важен как индикатор неблокирующего поведения:

```
2026-06-01 18:43:21.231 [reactor-http-nio-3] DEBUG r.netty.http.server.HttpServer - [c0fc3f5e,
L:/[0:0:0:0:0:0:1]:8085 - R:/[0:0:0:0:0:0:1]:51579] READ: 82B
...
2026-06-01 18:43:21.233 [reactor-http-nio-3] DEBUG r.netty.http.server.HttpServer - [c0fc3f5e-1,
L:/[0:0:0:0:0:0:1]:8085 - R:/[0:0:0:0:0:0:1]:51579] READ COMPLETE
```

**Про работу EventLoop в текущем контексте:**

- Поток `nio-3` лично переступает границу со Spring WebFlux через адаптер `ReactorHandlerAdapter`, заходит в Spring WebFlux слой, вызывает и выполняет метод нашего контроллера. Он входит в метод `test1()` контроллера, но в этот момент `ResponseEntity.ok(Map.of ("value", "Hello World"))` еще не создается, потому что controller

возвращает не готовый `ResponseEntity`, а `Mono`, который описывает будущую работу: через 3 секунды испустить сигнал -> после этого выполнить `map(...)` -> создать `ResponseEntity` -> передать результат дальше в HTTP response pipeline

- То есть поток `reactor-http-nio-3` не строит ответ сразу. Он получает от controller-а легкое описание будущей реактивной цепочки, Spring WebFlux подписывается на эту цепочку, а `Mono.delay(Duration.ofSeconds(3))` планирует time-based сигнал на Scheduler-е Project Reactor'a через 3 секунды, где по умолчанию `Mono.delay(...)` использует default parallel Scheduler в виде пула потоков.
- После этого `reactor-http-nio-3` освобождается. Он не ждет эти 3 секунды. Он возвращается обратно в EventLoop и может обрабатывать другие Channel-ы, другие HTTP-запросы и другие сетевые события.

```
2026-06-01 18:43:21.232 [reactor-http-nio-3] DEBUG r.netty.http.server.HttpServer - [c0fc3f5e-1,
L:/[0:0:0:0:0:0:1]:8085 - R:/[0:0:0:0:0:0:1]:51579] Handler is being applied:
org.springframework.http.server.reactive.ReactorHttpHandlerAdapter@51514798
2026-06-01 18:43:21.233 [reactor-http-nio-3] DEBUG r.netty.http.server.HttpServer - [c0fc3f5e-1,
L:/[0:0:0:0:0:0:1]:8085 - R:/[0:0:0:0:0:0:1]:51579] READ COMPLETE
```

### 7. Фаза ожидания и вывода: продолжение reactive pipeline после delay:

```
t0: reactor-http-nio-3 вызывает controller
controller возвращает Mono
Mono.delay планирует сигнал через 3 секунды
EventLoop освобождается

t0 + 3s: parallel-2 испускает сигнал 0L
map(...) выполняется
ResponseEntity создается
результат идет дальше в response pipeline
```

В логах это соответствует моменту, когда Reactor Netty видит финальный response frame снизу

```
2026-06-01 18:43:24.236 [parallel-2] DEBUG r.n.http.server.HttpServerOperations - [c0fc3f5e-1,
L:/[0:0:0:0:0:0:1]:8085 - R:/[0:0:0:0:0:0:1]:51579] Last HTTP response frame
2026-06-01 18:43:24.236 [parallel-2] DEBUG r.n.http.server.HttpServerOperations - [c0fc3f5e-1,
L:/[0:0:0:0:0:0:1]:8085 - R:/[0:0:0:0:0:0:1]:51579] Headers are not sent before onComplete()
```

Поток `parallel-2` продолжает reactive pipeline после задержки и участвует в формировании ответа. Но финальная запись в сетевой Channel должна пройти через Reactor Netty и EventLoop, потому что Channel обслуживается своим EventLoop-потоком.

```
parallel-2 продолжает reactive pipeline после delay
↓
создается ResponseEntity
↓
результат передается дальше в HTTP response pipeline
↓
сетевая запись возвращается в Reactor Netty / EventLoop
```

### 8. Сетевой выстрел байтами обратно клиенту:

```
2026-06-01 18:43:24.237 [reactor-http-nio-3] DEBUG r.netty.http.server.HttpServer - [c0fc3f5e-1,
L:/[0:0:0:0:0:0:1]:8085 - R:/[0:0:0:0:0:0:1]:51579] WRITE: 94B
```

```
|00000000| 48 54 54 50 2f 31 2e 31 20 32 30 30 20 4f 4b 0d |HTTP/1.1 200 OK.|
|00000010| 0a 43 6f 6e 74 65 6e 74 2d 54 79 70 65 3a 20 61 |.Content-Type: a|
...
```

### Главная мысль этого блока:

EventLoop не ждал 3 секунды.

Он быстро принял request, передал его в Spring WebFlux, получил reactive pipeline и освободился.

Через 3 секунды parallel-2 поток из пула Parallel Scheduler продолжил reactive chain, создал ResponseEntity, а финальная запись ответа вернулась в Reactor Netty / EventLoop, который сделал WRITE/FLUSH в Channel.

## 9. Очистка ресурсов и закрытие сокета:

```
2026-06-01 18:43:24.237 [reactor-http-nio-3] DEBUG r.n.http.server.HttpServerOperations - [c0fc3f5e-1,
L:/[0:0:0:0:0:0:0:1]:8085 - R:/[0:0:0:0:0:0:0:1]:51579] Decreasing pending responses count: 0
2026-06-01 18:43:24.237 [reactor-http-nio-3] DEBUG r.netty.http.server.HttpServer - [c0fc3f5e-1,
L:/[0:0:0:0:0:0:0:1]:8085 - R:/[0:0:0:0:0:0:0:1]:51579] FLUSH
2026-06-01 18:43:24.237 [reactor-http-nio-3] DEBUG r.n.http.server.HttpServerOperations - [c0fc3f5e-1,
L:/[0:0:0:0:0:0:0:1]:8085 - R:/[0:0:0:0:0:0:0:1]:51579] Last HTTP packet was sent, terminating the channel
2026-06-01 18:43:24.237 [reactor-http-nio-3] DEBUG r.netty.channel.ChannelOperations - [c0fc3f5e-1,
L:/[0:0:0:0:0:0:0:1]:8085 - R:/[0:0:0:0:0:0:0:1]:51579] [HttpServer] Channel inbound receiver cancelled
(subscription disposed).
2026-06-01 18:43:24.237 [reactor-http-nio-3] TRACE r.netty.channel.ChannelOperations - [c0fc3f5e,
L:/[0:0:0:0:0:0:0:1]:8085 - R:/[0:0:0:0:0:0:0:1]:51579] Disposing ChannelOperation from a channel
...
2026-06-01 18:43:24.237 [reactor-http-nio-3] TRACE r.netty.channel.ChannelOperations - [c0fc3f5e,
L:/[0:0:0:0:0:0:0:1]:8085 - R:/[0:0:0:0:0:0:0:1]:51579] Disposing ChannelOperation from a channel
java.lang.Exception: ChannelOperation terminal stack...
...
2026-06-01 18:43:24.238 [reactor-http-nio-3] DEBUG r.netty.http.server.HttpServer - [c0fc3f5e,
L:/[0:0:0:0:0:0:0:1]:8085 - R:/[0:0:0:0:0:0:0:1]:51579] READ COMPLETE
2026-06-01 18:43:24.238 [reactor-http-nio-3] DEBUG r.netty.http.server.HttpServer - [c0fc3f5e,
L:/[0:0:0:0:0:0:0:1]:8085 ! R:/[0:0:0:0:0:0:0:1]:51579] INACTIVE
2026-06-01 18:43:24.238 [reactor-http-nio-3] DEBUG r.netty.http.server.HttpServer - [c0fc3f5e,
L:/[0:0:0:0:0:0:0:1]:8085 ! R:/[0:0:0:0:0:0:0:1]:51579] UNREGISTERED
```

**Подробнее про FLUSH:** Когда поток nio-3 доходит до команды FLUSH, то механика работы состоит из 4 пунктов

1. Поток `nio-3` получает эти байты.
2. Он совершает системный вызов ядра ОС.
3. Ядро операционной системы забирает эти байты в свой сетевой стек и отправляет клиенту
4. Внутренний буфер Netty в Java-приложении снова становится пустым (очищенным).

**Подробнее про ошибку "`java.lang.Exception: ChannelOperation`" из логов:** Это не ошибка! Это некий трассировочный лог уровня TRACE. Reactor Netty специально генерирует искусственный `java.lang.Exception`, чтобы зафиксировать в логах полный стек-трейс реактивной цепочки.

```
...
2026-06-01 18:43:24.237 [reactor-http-nio-3] TRACE r.netty.channel.ChannelOperations - [c0fc3f5e,
L:/[0:0:0:0:0:0:0:1]:8085 - R:/[0:0:0:0:0:0:0:1]:51579] Disposing ChannelOperation from a channel
java.lang.Exception: ChannelOperation terminal stack
    at reactor.netty.channel.ChannelOperations.terminate(ChannelOperations.java:501)
    at reactor.netty.http.server.HttpServerOperations.terminateInternal(HttpServerOperations.java:1024)
    at reactor.netty.http.server.HttpServerOperations.operationComplete(HttpServerOperations.java:1019)
...
```

```
at reactor.netty.http.server.HttpTrafficHandler.flush(HttpTrafficHandler.java:342)
...
```

## 7.1. Концептуальный путь предыдущего запроса

1. Запрос приходит в WebFlux.
2. Controller вызывается на reactor-http-nio-.\*.
3. Controller быстро возвращает Mono.
4. Spring/WebFlux подписывается на Publisher(Mono).
5. Mono.delay(...) ставит timer на 3 секунды.
6. EventLoop освобождается.
7. Через 3 секунды default parallel Scheduler испускает сигнал 0L.
8. Тот же parallel поток выполняет map(...), который вызывается после delay(после 3 секунд ожидания).
9. createResponse() создает ResponseEntity.
10. ResponseEntity передается в Spring WebFlux response-handling слой.
11. Spring WebFlux превращает ResponseEntity в HTTP status, headers и body.
12. Reactor Netty получает готовый HTTP response и возвращает запись в EventLoop.
13. Финальный WRITE/FLUSH в Channel делает Netty/EventLoop.

## 8. Резюмируя (Аналогия с рестораном)

**EventLoop:** — это официант у столика. Он должен принять у клиента заказ (запрос) и уйти обслуживать другие столики (других клиентов).

**Parallel пул:** — это повара на кухне. Они взяли заказ, таймер отщелкал время приготовления (3 секунды), повар приготовил блюдо и отдал на тарелке клиенту (Spring сформировал JSON).

**Финальная мысль:** Повар (parallel) не идет сам в зал к клиенту. Он зовет оригинального официанта (EventLoop). Официант берет готовую тарелку и физически ставит её на стол гостю (пишет байты в сокет).

## 9. Резюмируя некоторые мысли при вызове контроллера

После запуска endpoint-а важно понимать:

1. Controller вызван не "сам по себе".  
До него запрос прошел через Reactor Netty и Spring WebFlux.
2. Thread name reactor-http-nio-\* показывает,  
что код выполняется на Reactor Netty event-loop thread.

## 10. Главный вывод текущего шага

На уровне ментальной модели можно сказать так:

В Spring MVC запрос входит в Servlet container, получает worker thread и дальше идет к DispatcherServlet.

- 1) В Spring WebFlux запрос входит в Reactor Netty,
- 2) попадает в Netty Channel,
- 3) обрабатывается в EventLoop,
- 4) проходит ChannelPipeline
- 5) и только потом попадает в Spring WebFlux путь запроса.

## 11. Весь концептуальный план обработки запроса для будущих уроков

```

Netty server
  ↓
Channel
  ↓
ChannelPipeline
  ↓
EventLoopGroup / EventLoop
  ↓
Reactor Netty HttpServer
  ↓
ReactorHttpHandlerAdapter
  ↓
Spring HttpHandler
  ↓
WebHandler / WebFilter chain
  ↓
DispatcherHandler
  ↓
HandlerMapping
  ↓
HandlerAdapter
  ↓
Controller returns Mono/Flux
  ↓
HandlerResult
  ↓
HandlerResultHandler
  ↓
Spring subscribes / writes Publisher result
  ↓
Reactor Netty writes HTTP response
  ↓
Netty Channel write/flush
  
```

Краткий итог:

Netty server — это нижний runtime-движок WebFlux-приложения. Он принимает соединения и сетевые события.

Spring WebFlux — это framework-слой, который получает уже адаптированный HTTP request и решает, какой controller вызвать.

## 12. Возвращаясь к Channel: соединение как объект внутри Netty

Теперь нам нужно вернуться к уже знакомой сущности - Channel.

Channel — это объект Netty, который представляет конкретное сетевое соединение или сетевой канал.

## 12.1 Channel — это не HTTP-запрос

Не надо думать так:

```
1 HTTP request = 1 Channel
```

Правильнее:

```
1 TCP connection = 1 Channel
```

А уже внутри одного TCP-соединения может пройти один или несколько HTTP-запросов.

Например, если клиент сделал один запрос и соединение останется открытым, то одно соединение может быть переиспользовано:

```
Client
  ↓
TCP connection
  ↓
Channel-A
  ↓
GET /test1
  ↓
response
  ↓
GET /test2
  ↓
response
  ↓
GET /test3
  ↓
response
```

То есть:

```
Channel живет на уровне соединения,
а HTTP request живет на уровне протокола HTTP внутри этого соединения.
```

## 12.2. Как Channel виден в логах

В логах Reactor Netty мы видели примерно такую запись:

```
[c0fc3f5e, L:[0:0:0:0:0:0:1]:8085 - R:[0:0:0:0:0:0:1]:51579]
```

`c0fc3f5e` — имя/номер конкретного сетевого соединения внутри Netty.

L — local address. То есть адрес нашего приложения.

R — remote address. То есть адрес клиента.

То есть:

```
Channel c0fc3f5e:
  локальная сторона: localhost:8085
  удаленная сторона: localhost:51579
```

То есть, это конкретное соединение между клиентом и нашим сервером

### 12.3. У Channel есть жизненный цикл

Channel не появляется навсегда.

Упрощенный жизненный цикл:

```
new connection
  ↓
Channel created
  ↓
REGISTERED
  ↓
ACTIVE
  ↓
READ
  ↓
READ COMPLETE
  ↓
WRITE
  ↓
FLUSH
  ↓
INACTIVE
  ↓
UNREGISTERED
```

Теперь разберем эти состояния человеческим языком.

**REGISTERED** - Channel зарегистрирован на EventLoop

**ACTIVE** - Channel стал активным

**READ** - EventLoop прочитал входящие байты из Channel Например, клиент отправил:

```
GET /test1 HTTP/1.1
Host: localhost:8085
```

Для Netty это сначала просто набор байтов вида `47 45 54 20 2f 74 65 73 74 31 ...`

Потом эти байты проходят через ChannelPipeline, где HTTP codec превращает их в HTTP request.

Но на уровне Channel мысль такая: в этот Channel пришли данные, EventLoop их прочитал

**READ COMPLETE** - Текущая read-фаза завершена.

READ COMPLETE не означает, что весь HTTP-запрос уже полностью обработан бизнес-логикой.

И не означает, что response уже отправлен клиенту.

Оно означает только то, что EventLoop на текущей итерации завершил чтение доступных входящих данных из Channel.



**WRITE - Netty записывает данные ответа в Channel** На этом этапе HTTP response уже превращается в байты.

Например

```
HTTP/1.1 200 OK
Content-Type: application/json
...
{"value":"Hello World"}
```

Для Netty это снова набор байтов, который нужно отправить клиенту - **WRITE: 94B**. То есть, в Channel записали 94 байта HTTP-ответа.

### **FLUSH и WRITE - запись ответа**

Можно думать так:

WRITE — положить данные в выходной буфер Netty.

FLUSH — протолкнуть накопленные данные дальше, чтобы они реально ушли в socket/OC.

То есть, технически:

WRITE подготавливает outbound данные в байтах,  
FLUSH заставляет Netty протолкнуть их дальше в сторону сокета.

**INACTIVE - Channel больше не активен**

**UNREGISTERED - Channel снят с EventLoop**

## 12.4. Channel можно представить как “трубу связи”

По этой трубе данные идут в обе стороны, но сама труба не знает бизнес-смысл запросов, ведь Channel ничего не знает о том, что

```
это запрос на /profile/123 эндпоинт
это пользователь 123
это надо сходить в БД
это надо вернуть JSON
```

Channel знает только:

```
соединение активно
пришли байты
байты прочитали
надо записать байты ответа
соединение закрыто
```

## 12.5. Итоговые мысли для вывода:

Channel — это Netty-объект, представляющий сетевое соединение.

Channel не равен HTTP-запросу.

Один Channel может обработать один или несколько HTTP-запросов, если соединение живет дольше, чем один request-response обмен.

Channel имеет жизненный цикл:

REGISTERED -> ACTIVE -> READ -> READ COMPLETE -> WRITE -> FLUSH -> INACTIVE -> UNREGISTERED.

Channel закрепляется за EventLoop.

EventLoop может обслуживать много Channel-ов.

Если заблокировать EventLoop, страдают все Channel-ы, которые он обслуживает.

Входящие запросы приходят через inbound Channel.

Внешние вызовы из нашего приложения идут через outbound Channel.

Следующий шаг:

разобрать ChannelPipeline — цепочку обработчиков, через которую байты превращаются в HTTP request и обратно в HTTP response.

## 13. ChannelPipeline

Мы уже разобрали, что Channel — это объект Netty, который представляет конкретное сетевое соединение.

Теперь возникает следующий вопрос:

Окей, в Channel пришли байты.

Но кто именно превращает эти байты в HTTP request?

Кто логирует сетевые события?

Кто кодирует HTTP response обратно в байты?

Кто передает управление дальше в Reactor Netty?

Все это как раз и делает ChannelPipeline

Если объяснять на пальцах: ChannelPipeline — это цепочка обработчиков, которая принадлежит к конкретному Channel. То есть у каждого сетевого соединения есть не только сам Channel, но и свой конвейер обработки:

Концептуально:

```
TCP connection
  ↓
Channel
  ↓
ChannelPipeline
  ↓
Handler-ы
```

Можно представить так:

```
Channel c0fc3f5e
```

```

ChannelPipeline
|
| [loggingHandler]
|   ↓
| [httpCodec]
|   ↓
| [httpTrafficHandler]
|   ↓
| [reactiveBridge]

```

Главная идея:

Channel — это "труба связи". ChannelPipeline — это "линия обработки" внутри этой трубы.

### 13.1. Зачем нужен ChannelPipeline

Сетевое соединение само по себе приносит в приложение просто байты вида `47 45 54 20 2f 74 65 73 74 31 20 48 54 54 50 2f ...`

Эти байты нужно обработать:

```

сырые байты
  ↓
распарсить HTTP-протокол
  ↓
понять и спарсить method/path/headers/body
  ↓
поднять событие на уровень Reactor Netty
  ↓
передать дальше в Spring WebFlux

```

Именно это делает ChannelPipeline через набор обработчиков

### 13.2. Handler: один шаг в конвейере

Каждый элемент ChannelPipeline'a называется ChannelHandler. Где ChannelHandler — это один обработчик внутри цепочки.

Один такой handler может:

- залогировать событие;
- декодировать байты;
- закодировать объект обратно в байты;
- обработать ошибку;
- передать событие дальше;
- изменить данные;
- инициировать запись ответа;
- связать Netty с Reactor Netty.

Например, в логах из примера выше есть вот такой:

```

DefaultChannelPipeline{
  (reactor.left.loggingHandler = ReactorNettyLoggingHandler),

```

```
(reactor.left.httpCodec = HttpServerCodec),
(reactor.left.httpTrafficHandler = HttpTrafficHandler),
(reactor.right.reactiveBridge = ChannelOperationsHandler)
}
```

Разберем по смыслу:

```
loggingHandler - логирует сетевые события: REGISTERED, ACTIVE, READ, WRITE, FLUSH и т.д.

httpCodec - умеет декодировать HTTP request из байтов и кодировать HTTP response обратно в байты.

httpTrafficHandler - помогает Reactor Netty управлять HTTP-трафиком, request-response обменом и особенностями
HTTP-сервера.

reactiveBridge - это мост, через который входящие read-события из Netty попадают в Reactor Netty,
а исходящие write-события из Reactor Netty возвращаются обратно в Netty pipeline.
```

На этом этапе не надо знать внутренности каждого handler-а. Важно понять главное:

```
Pipeline - это цепочка маленьких инфраструктурных обработчиков, через которую проходит каждое событие Channel-а.
```

### 13.3. ChannelPipeline "на пальцах":

Когда клиент отправляет HTTP-запрос, данные идут в сторону приложения. Это называется inbound-направление.

```
Клиент отправил байты.
Pipeline прогнал их через обработчики.
HTTP codec понял, что это GET /test1.
Reactor Netty поднял это до HTTP request/response модели.
Spring WebFlux получил запрос и пошел искать controller.
```

### 13.4. Где здесь Reactor Netty

ChannelPipeline сам по себе не знает про @RestController.

Для Netty вся работа — это:

```
Channel
ChannelPipeline
ChannelHandler
ByteBuf
HTTP codec
read/write events
```

А Spring WebFlux живет уровнем выше:

```
ServerWebExchange
HandlerMapping
HandlerAdapter
Controller
Mono/Flux
```

Между ними нужен мост. И в наших pipeline'ах таким мостом является reactiveBridge=ChannelOperationsHandler.

## 14. EventLoopGroup и EventLoop: кто исполняет ChannelPipeline

Мы уже разобрали три важные сущности: Netty server, Channel и ChannelPipeline

Теперь мы должны ответить на вопрос:

Кто физически выполняет обработчики ChannelPipeline?

```
- EventLoop поток
```

То есть,

```
Кто вызывает loggingHandler?
Кто вызывает httpCodec?
Кто вызывает httpTrafficHandler?
Кто вызывает reactiveBridge?
Кто делает READ?
Кто делает WRITE/FLUSH?
```

```
Ответ - EventLoop поток
```

EventLoopGroup же — это группа таких EventLoop-потоков.

```
EventLoopGroup
├── EventLoop-1
├── EventLoop-2
├── EventLoop-3
└── EventLoop-4
```

А EventLoop — это конкретный поток внутри этой группы.

```
Spring Boot WebFlux приложение стартует.
↓
Reactor Netty поднимает HTTP server.
↓
Под сервер создается EventLoopGroup.
↓
Внутри EventLoopGroup есть несколько EventLoop-потоков.
↓
Эти потоки обслуживают сетевые Channel-ы.
```

На пальцах:

```
EventLoop:
1. ждет сетевые события, то есть крутится в вечном цикле вокруг селектора;
2. узнает, какой Channel готов к чтению или записи;
3. выполняет ChannelPipeline для этого Channel;
4. выполняет небольшие отложенные задачи;
5. снова возвращается к ожиданию событий.
```

## 14.1 EventLoopGroup, boss и worker

В Netty часто говорят о двух группах: Boss EventLoopGroup и Worker EventLoopGroup

Упрощенно:

```
Boss EventLoopGroup - принимает новые соединения
```

```
Worker EventLoopGroup -  
  обслуживает уже принятые соединения:  
  READ, WRITE, FLUSH, ChannelPipeline
```

Концепция работы:

```
Client connects  
  ↓  
Boss EventLoop принимает соединение  
  ↓  
создается Channel для Worker EventLoopGroup  
  ↓  
этот новый Channel регистрируется на Worker EventLoop  
  ↓  
Worker EventLoop обслуживает read/write этого Channel
```

Нам не нужно руками создавать boss/worker группы. Это настраивает Reactor Netty под капотом.

## 15. Финал урока

Сегодня мы дошли до границы Reactor Netty:

```
Client  
  ↓  
Netty server  
  ↓  
Channel  
  ↓  
ChannelPipeline  
  ↓  
EventLoopGroup / EventLoop  
  ↓  
reactiveBridge / Reactor Netty - дальше данные идут каким-то образом в Spring Webflux
```

## 16. Следующий урок — Spring WebFlux Request Path поверх Reactor Netty

Цель: Понять, как request после Reactor Netty попадает в Spring WebFlux и как результат controller-а превращается в HTTP response.

Темы:

```
1. Reactor Netty  
2. ReactorHttpHandlerAdapter  
3. HttpHandler
```

```
4. WebHandler / WebFilter chain
5. DispatcherHandler
6. HandlerMapping
7. HandlerAdapter
8. Controller returns Mono/Flux
9. HandlerResult
10. HandlerResultHandler
11. Где происходит subscribe
12. Response write path
```

Будем изучать концептуальную модель:

## Spring-Webflux

```
Reactor Netty
↓
ReactorHttpHandlerAdapter
↓
HttpHandler
↓
WebHandler
↓
DispatcherHandler
↓
HandlerMapping
↓
HandlerAdapter
↓
Controller
↓
Mono/Flux
↓
response write
```